

SKILL: forecast-dashboard-tech

Código, estructura de datos y fórmulas del Dashboard Forecast.

Rutas (bash VM)

- Plantilla: /sessions/*/mnt/Forecast/generador/plantilla.html (editar acá)
- HTML final: /sessions/*/mnt/Forecast/Dashboard_Forecast.html (generado; no editar)
- Generador: /sessions/*/mnt/Forecast/generador/generar_dashboard.py
- Cache: /sessions/*/mnt/Forecast/generador/historico_vp.json
- **Archivos congelados (histórico):** el generador (build_historico) los busca en la subcarpeta **Forecast\Histórico** (variantes con/sin tilde), NO en la carpeta principal. Nombre Forecast MM-AA.xlsm (ej. Forecast 07-26.xlsm).
- Los Excel viven en Google Drive (G:); puede que no monten en bash → operar vía el .bat en la PC.

Estructura Forecast.xlsm (hoja Forecast)

- Fila 3 = encabezados; datos por código desde fila 6.
- Bloques mensuales **stride 14 desde col CJ (88)**: Venta prom. base(+0), VP unidades(+1), VP USD(+4), Stock inicio(+5), Meses(+7), Cant. proyectada(+8), Cant. confirmada(+10).
- Identidad: UN=A, GA=B, Código=C, Categoría=F, Meses/Cat=H, Stock=BN, Meses=BO, Tendencia=AN, **Precio ponderado/MIX=AS**.
- Venta real por mes: **AB-AM** (fila 3 = etiqueta de mes, puede venir como fecha → parsear datetime; usar valor absoluto).
- Estacionalidad/Evento/TC del mes en fila 2 del bloque (offsets +9, +12, +4).
- Cada GA tiene 2 filas de encabezado con banner IMPO (estado, número, disponible, cantidades, USD PI/CI).

Estructura Costos.xlsm (hoja General)

Por código: Costo=AC, Lista 1=J, Moneda=I, CMM=F (referencia, NO se usa), Q(17)=lista futura.

Fórmulas

- **Exceso (método B):** $\text{ExcesoU} = \text{Stock} - \sum \text{VP}(\text{meses del Ideal}); \text{factor} \times N$ extiende el horizonte (ideal×N); solo exceso > 0.
- **CMM** = $(J \times 0.7 \times 0.93 - \text{Costo}) / (J \times 0.7)$.
- **CMM Liq.** = $(J \times (1 - \text{DtoLiq}) \times 0.7 \times 0.93 - \text{Costo}) / (J \times (1 - \text{DtoLiq}) \times 0.7)$.
- **Dto. Máx.** = mayor dto con contribución ≥ mínimo (def 10%). **Dto. Liq.** = Máx - gap (def 5%).
- **Lista Base**=J · **Lista Mix**=AS · **Lista Dto. Máx.** = $J \times 0.7 \times (1 - \text{DtoMáx})$ (PV mínimo).

- **Costo total**=Costo×ExcesoU · **Mix total**=ListaMix×ExcesoU · **Mix total Liq. (Obj. Liq.)**=ListaMix×(1-DtoLiq)×ExcesoU.
- **Proyección:** VA c/stk = valor de “V.Ajust. c/stock” (sin ×estacionalidad); CMV=Costo×VPu; CMV/Vta=CMV/Venta.
- **Histórico:** VPu=CJ+1 del archivo; VRu=AB-AM (abs); VP\$=MIX×VPu; VR\$=venta real facturada; %=(real-proy)/proy.

Archivo de venta real (V.R. mensual)

Alimenta la columna **V.R. \$-USD** del Histórico. Se arma desde Libro1.xlsx (export de facturación SAP). Columnas: Fecha=G, Código=O, Grupo=P, Cantidad=Q, Total S/Desc.=U, TC=W.

- **Criterio (una sola columna \$-USD):** por línea, si TC>0 (Importados) → USD = Total/TC; si TC=0/None (Nacionales) → pesos = Total. Se suma por código y por mes.
- **Formato:** fila 1 = Grupo / Código / Cantidad tot. / VR \$-USD + rótulo de mes; fila 2 = Cant / \$-USD; datos desde fila 3.
- **read_realusd:** detecta el archivo con los patrones Venta_real*, Venta real*, V.R*, VR*, V R*; y toma como columna de valor la subcolumna cuyo rótulo **contiene** “usd” (if “usd” in sub), lo que acepta \$-USD y USD. El código está en la col con “digo” en fila 1.

Lectura de Excel (openpyxl)

Usar openpyxl.load_workbook(path, read_only=True, data_only=True) por el tamaño del .xlsm. No pre-redondear: guardar los valores con precisión (función N() en el generador) para que coincidan con Excel.

Patrón de parche a la plantilla

```
import sys
p=sys.argv[1]; c=open(p,encoding='utf-8').read()
def R(o,n):
    global c
    assert c.count(o)==1, ("NO UNICO: "+repr(o[:55]))+" | "+str(c.count(o)))
    c=c.replace(o,n,1)
R("<string exacto viejo>", "<nuevo>")
open(p,'w',encoding='utf-8').write(c)
print("patch OK")
```

Reglas: anclas EXACTAS (respetar var(--azul) etc., no hardcodear hex si el código usa variables). Si assert falla, no se escribe nada → corregir el ancla. Un cambio grande que abarca varias funciones: recortar por índices c.index(inicio)/c.index(fin) con cuidado de no comerse funciones vecinas (ej. no borrar renderLiqCharts al reemplazar renderLiqPreview).

Validación (obligatoria antes de cerrar)

extraer el <script> de la plantilla, reemplazar el placeholder de datos y chequear sintaxis
python3 -c "import re;h=open('plantilla.html',encoding='utf-8').read();\n"

```
s=re.search(r'<script>(.*?)</script>',h,re.S).group(1).replace('/*_DATA_*/','null');  
\nopen('/tmp/chk.js','w').write(s)"  
node --check /tmp/chk.js && echo "JS OK"
```

Para lógica numérica: probar en Node con stubs de DATA/DOM (excU, CMM Liq, Obj Liq, totales, % de gráficos, ventana de meses) antes de publicar.

Panel de pedidos IMPO (renderPanel)

- Las tarjetas con **número de pedido** salen del banner IMPO del GA (DATA.impo[ga][m], número en H1 offset +10).
- Los meses con **compra proyectada pero sin número** (número vacío tipo “IMPO-”) muestran una tarjeta **“Proyectado”**: por GA en scope se suma mval(r,m,2) (columna COMPRA / cant. confirmada, índice 2) de las filas visibles, solo si el GA no tiene número ese mes. Así el panel queda consistente con la columna COMPRA de la tabla. Si no hay ni número ni compra proyectada → “Sin pedidos”.

Rendimiento

- Cache Histórico (historico_vp.json): los “Forecast MM-AA” se abren 1 vez; un mes nuevo se detecta y se agrega solo.
- visibleRows memoizado; debounce en el buscador de filtros; cacheo por pestaña; ventana de 4 meses por vista.